Karnatak Law Society's

GOGTE INSTITUTE OF TECHNOLOGY,

An Autonomous Institution under to Visvesraya Technological University, Belagavi. Udyambag, Belgaum-590008

(Recognised by A.I.C.T.E. New Delhi)

DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

Academic year 2024-2025

Seventh Semester

IoT and Data Acquisition Laboratory Manual (22EE61)

Staff Incharge

i) Dr. Sudhakar C.J

ii) Dr. Ramesh B

Our Vision

Department of Electrical and Electronics Engineering focuses on Training Individual aspirants for Excellent Technical aptitude, performance with outstanding executive caliber and industrial compatibility.

Our Mission

To impart optimally good quality education in academics and real time work domain to the students to acquire proficiency in the field of Electrical and Electronics Engineering and to develop individuals with a blend of managerial skills, positive attitude, discipline, adequate industrial compatibility and noble human values.

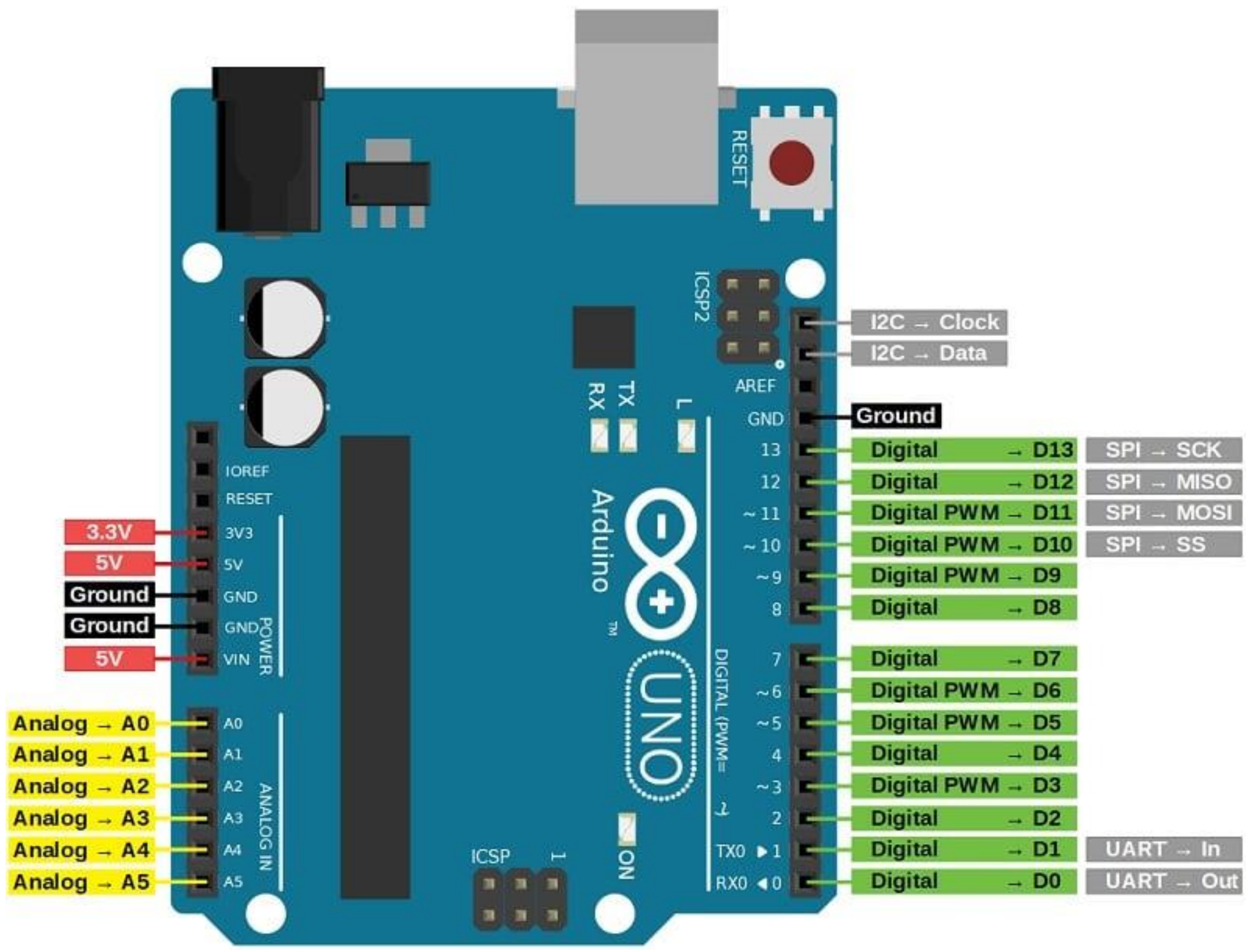
Index Sheet

Expt. No.:	Date	Title of the experiment	Marks (Max 20)	Faculty Signature
1		Real-time monitoring and measurement of weather data		
2		Relay based real-time control of electrical equipment’s.		
3		Water level monitoring with buzzer		
4		Automatic temperature controlling system		
5		Flame detection and alerting system		
6		Automatic street lighting system		
7		LED blinking operation using MATLAB		
8		MATLAB Data Logging, Analysis and Visualization: Plotting DHT11 Sensor readings on MATLAB		
9				

Introduction to Arduino Uno (R3)

Arduino boards are programmed using a language derived from C and C++ in Arduino's Integrated Development Environment (IDE). Arduino Uno is based on the ATmega328 by Atmel. The Arduino Uno pinout consists of 14 digital pins, 6 analog inputs, a power jack, USB connection and ICSP header.

Arduino Uno is the most standard board available and probably the best choice for a beginner. We can directly connect the board to the computer via a USB Cable which performs the function of supplying the power as well as acting as a serial port.



Vin: This is the input voltage pin of the Arduino board used to provide input supply from an external power source.

5V: This pin of the Arduino board is used as a regulated power supply voltage and it is used to give supply to the board as well as onboard components.

3.3V: This pin of the board is used to provide a supply of 3.3V which is generated from a voltage regulator on the board

GND: This pin of the board is used to ground the Arduino board.

Reset: This pin of the board is used to reset the microcontroller. It is used to Resets the microcontroller.

Analog Pins: The pins A0 to A5 are used as an analog input and it is in the range of 0-5V.

Digital Pins: The pins 0 to 13 are used as a digital input or output for the Arduino board.

Serial Pins: These pins are also known as a UART pin. It is used for communication between the Arduino board and a computer or other devices. The transmitter pin number 1 and receiver pin number 0 is used to transmit and receive the data resp.

External Interrupt Pins: This pin of the Arduino board is used to produce the External interrupt and it is done by pin numbers 2 and 3.

PWM Pins: This pins of the board is used to convert the digital signal into an analog by varying the width of the Pulse. The pin numbers 3,5,6,9,10 and 11 are used as a PWM pin.

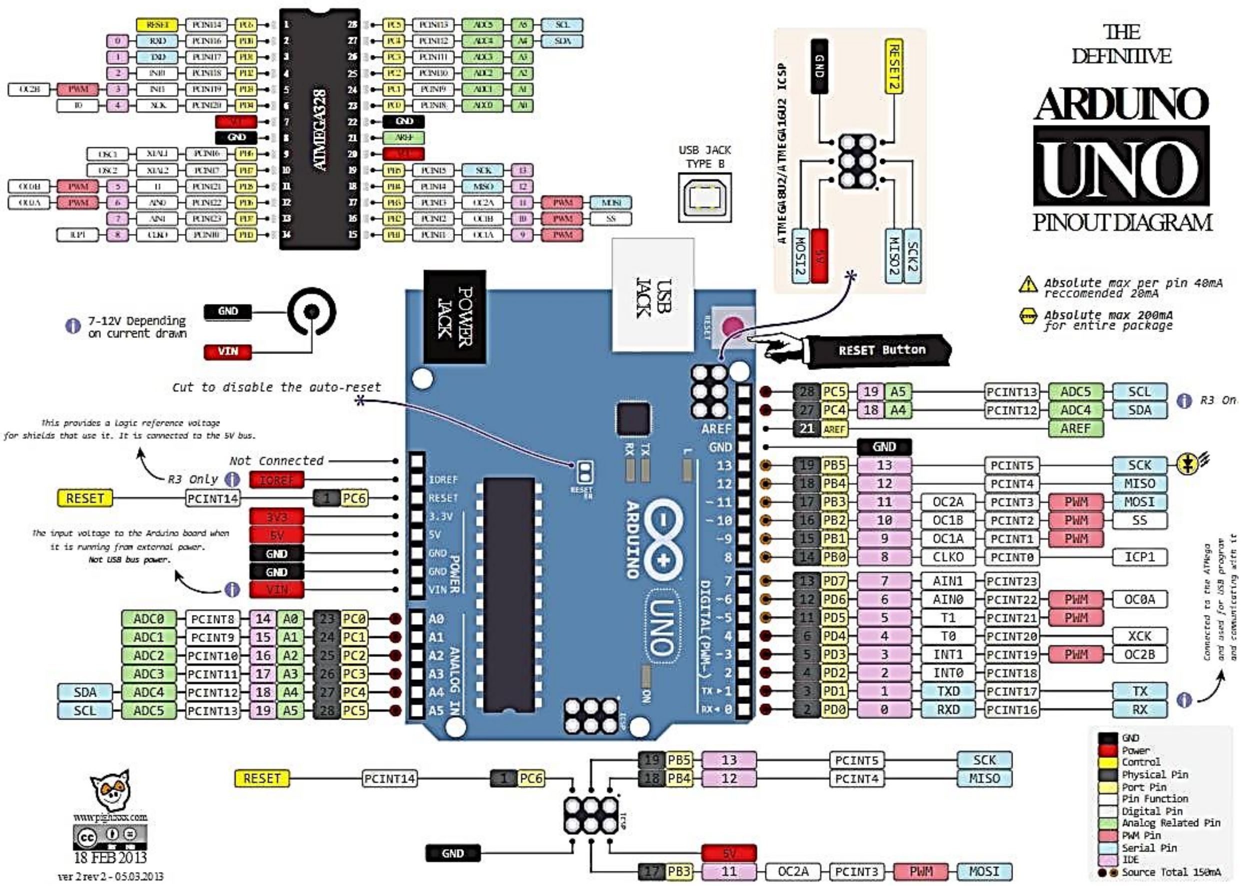
SPI Pins: This is the Serial Peripheral Interface pin, it is used to maintain SPI communication with the help of the SPI library. SPI pins include:

1. SS: Pin number 10 is used as a Slave Select
2. MOSI: Pin number 11 is used as a Master Out Slave In
3. MISO: Pin number 12 is used as a Master In Slave Out
4. SCK: Pin number 13 is used as a Serial Clock

LED Pin: The board has an inbuilt LED using digital pin-13. The LED glows only when the digital pin becomes high.

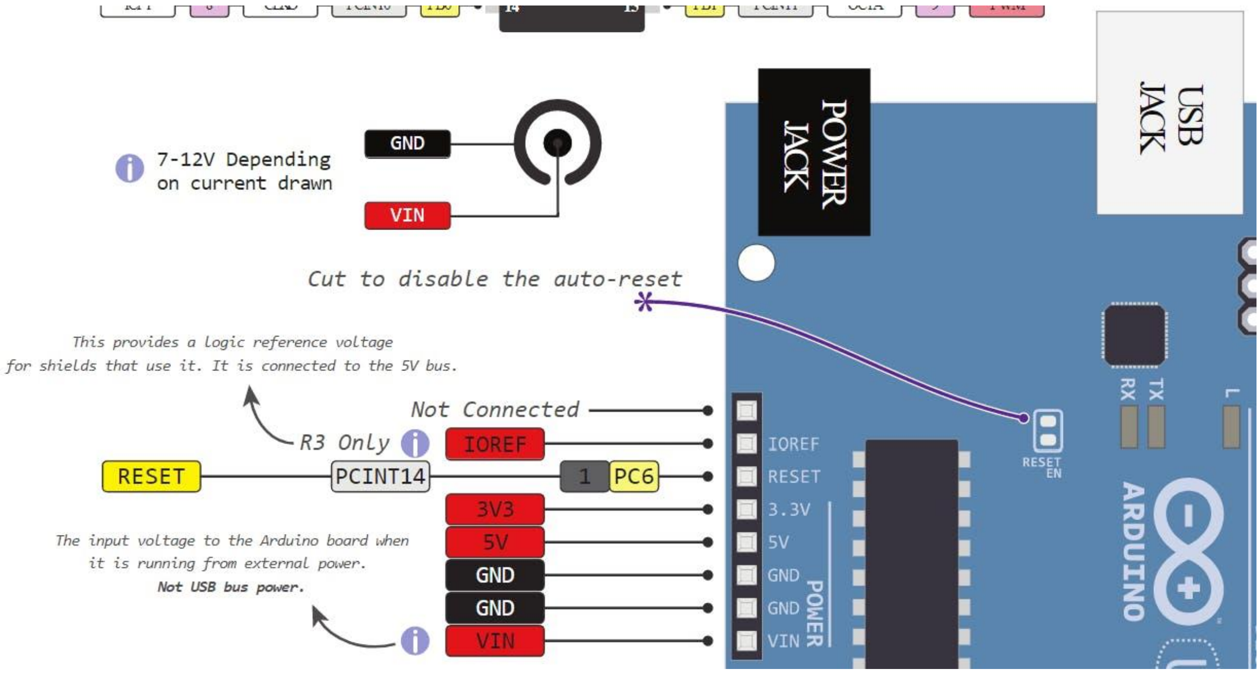
AREF Pin: This is an analog reference pin of the Arduino board. It is used to provide a reference voltage from an external power supply.

Detailed Arduino Uno pinout - Power Supply



There are 3 ways to power the Arduino Uno:

- **Barrel Jack** - The Barrel jack, or DC Power Jack can be used to power your Arduino board. The barrel jack is usually connected to a wall adapter. The board can be powered by 5-20 volts but the manufacturer recommends to keep it between 7-12 volts. Above 12 volts, the regulators might overheat, and below 7 volts, might not suffice.
- **VIN Pin** - This pin is used to power the Arduino Uno board using an external power source. The voltage should be within the range mentioned above.
- **USB cable** - when connected to the computer, provides 5 volts at 500mA.



There is a polarity protection diode connecting between the positive of the barrel jack to the VIN pin, rated at 1 Ampere.

The power source you use determines the power you have available for your circuit. For instance, powering the circuit using the USB limits you to 500mA. Take into consideration that this is also used for powering the MCU, its peripherals, the on-board regulators, and the components connected to it. When powering your circuit through the barrel jack or VIN, the maximum capacity available is determined by the 5 and 3.3 volts regulators on-board the Arduino.

- **5v and 3v3**

They provide regulated 5 and 3.3v to power external components according to manufacturer specifications.

- **GND**

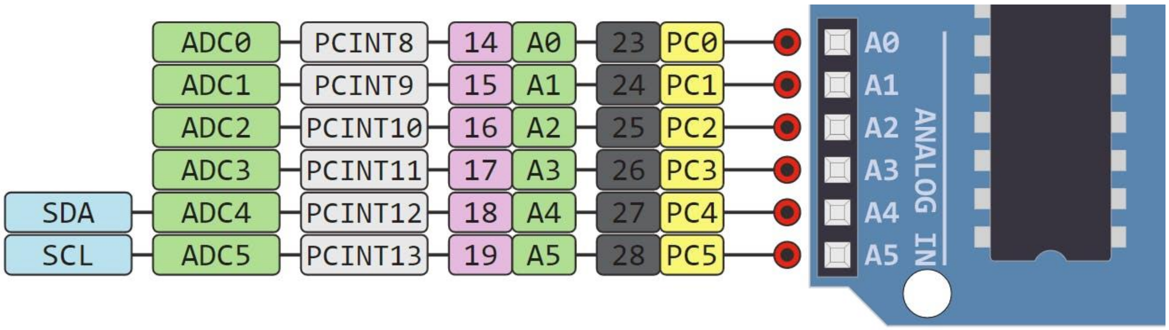
In the Arduino Uno pinout, you can find 5 GND pins, which are all interconnected.

The GND pins are used to close the electrical circuit and provide a common logic reference level throughout your circuit. Always make sure that all GNDs (of the Arduino, peripherals and components) are connected to one another and have a common ground.

- **RESET** - resets the Arduino
- **IOREF** - This pin is the input/output reference. It provides the voltage reference with which the microcontroller operates.

Arduino Uno Pinout - Analog IN

The Arduino Uno has 6 **analog pins**, which utilize ADC (Analog to Digital converter). These pins serve as analog inputs but can also function as digital inputs or digital outputs.



Analog to Digital Conversion

ADC stands for Analog to Digital Converter. ADC is an electronic circuit used to convert analog signals into digital signals. This digital representation of analog signals allows the processor (which is a digital device) to measure the analog signal and use it through its operation.

Arduino Pins A0-A5 are capable of reading analog voltages. On Arduino the ADC has 10-bit resolution, meaning it can represent analog voltage by 1,024 digital levels. The ADC converts voltage into bits which the microprocessor can understand.

One common example of an ADC is Voice over IP (VoIP). Every smartphone has a microphone that converts sound waves (voice) into analog voltage. This goes through the device's ADC, gets converted into digital data, which is transmitted to the receiving side over the internet.

Arduino Uno Pinout - Digital Pins

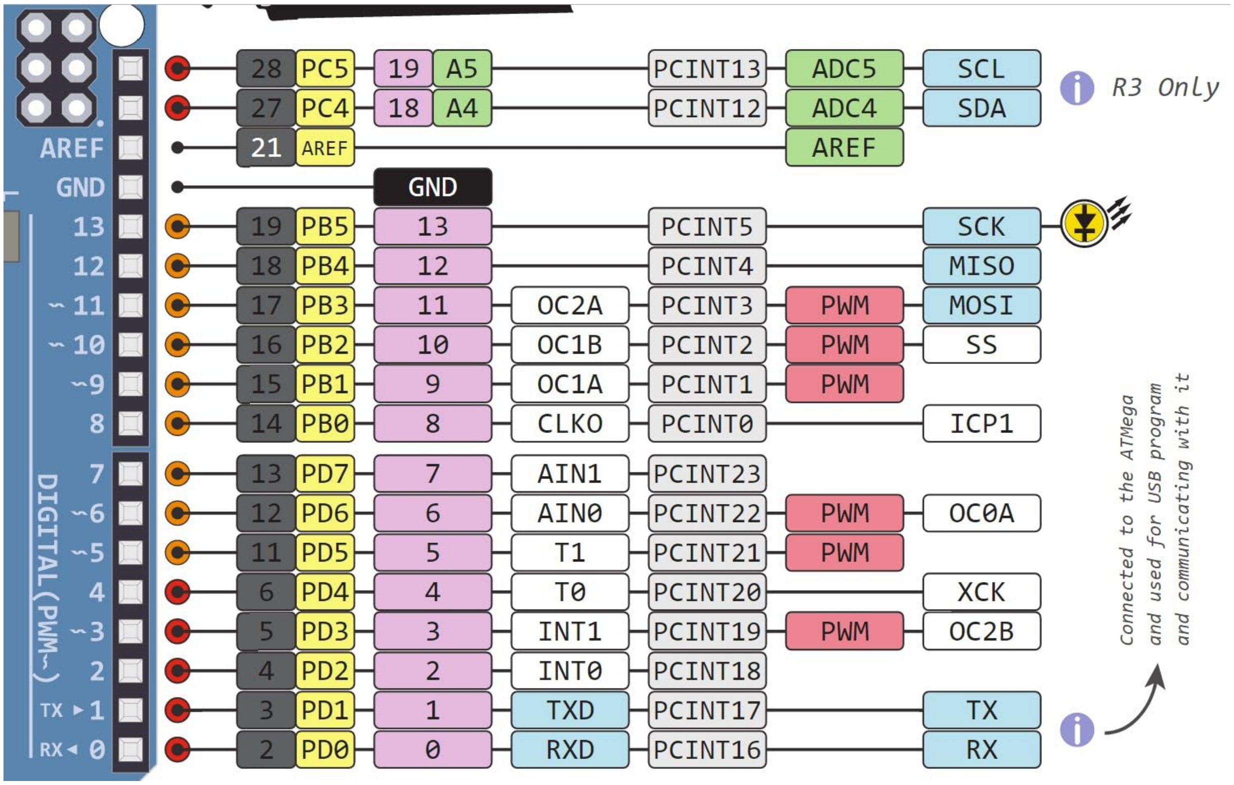
Pins 0-13 of the Arduino Uno serve as digital input/output pins.

Pin 13 of the Arduino Uno is connected to the built-in LED.

In the Arduino Uno - pins 3,5,6,9,10,11 have PWM capability.

It's important to note that:

- Each pin can provide/sink up to 40 mA max. But the recommended current is 20 mA.
- The absolute max current provided (or sank) from all pins together is 200mA



What does digital mean?

Digital is a way of representing voltage in 1 bit: either 0 or 1. Digital pins on the Arduino are pins designed to be configured as inputs or outputs according to the needs of the user. Digital pins are either on or off. When ON they are in a HIGH voltage state of 5V and when OFF they are in a LOW voltage state of 0V.

On the Arduino, When the digital pins are configured as **output**, they are set to 0 or 5 volts.

When the digital pins are configured as **input,** the voltage is supplied from an external device. This voltage can vary between 0-5 volts which is converted into digital representation (0 or 1). To determine this, there are 2 thresholds:

- Below 0.8v - considered as 0.
- Above 2v - considered as 1.

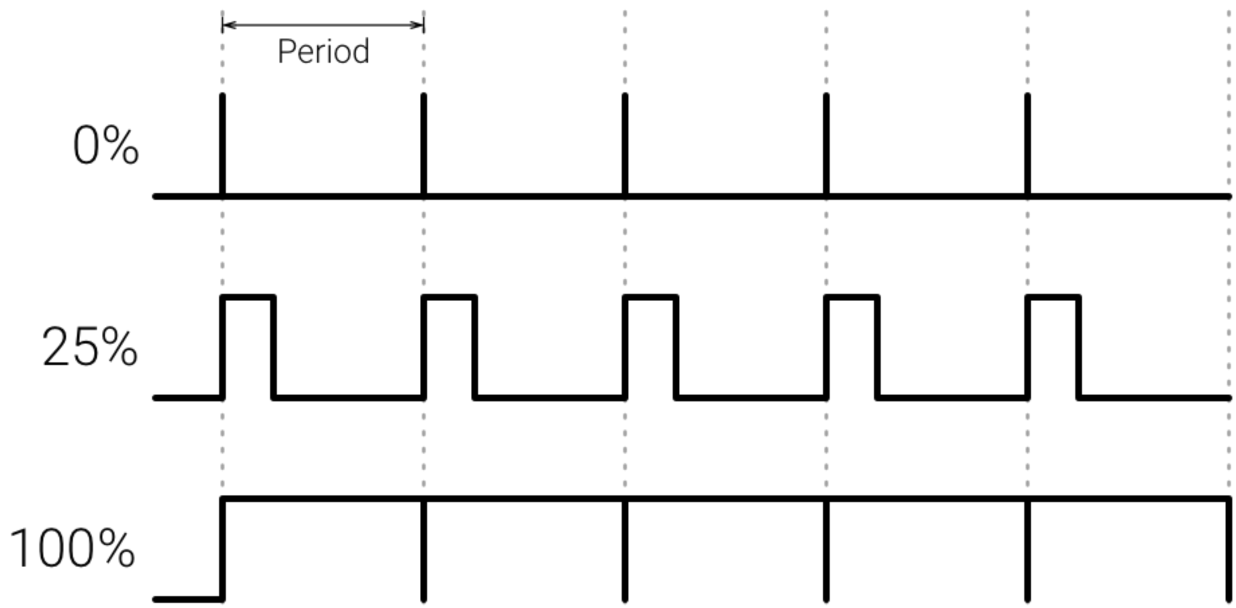
When connecting a component to a digital pin, make sure that the logic levels match. If the voltage is in between the thresholds, the returning value will be undefined.

What is PWM?

In general, Pulse Width Modulation (PWM) is a modulation technique used to encode a message into a pulsing signal. A PWM is comprised of two key components: **frequency** and **duty**

cycle. The PWM frequency dictates how long it takes to complete a single cycle (period) and how quickly the signal fluctuates from high to low. The duty cycle determines how long a signal stays high out of the total period. Duty cycle is represented in percentage.

In Arduino, the PWM enabled pins produce a constant frequency of ~ 500Hz, while the duty cycle changes according to the parameters set by the user. See the following illustration:



PWM signals are used for speed control of DC motors, dimming LEDs and more.

Communication Protocols

Serial (TTL) - Digital pins 0 and 1 are the serial pins of the Arduino Uno.

They are used by the onboard USB module.

What is Serial Communication?

Serial communication is used to exchange data between the Arduino board and another serial device such as computers, displays, sensors and more. Each Arduino board has at least one serial port. Serial communication occurs on digital pins 0 (RX) and 1 (TX) as well as via USB. Arduino supports serial communication through digital pins with the SoftwareSerial Library as well. This allows the user to connect multiple serial-enabled devices and leave the main serial port available for the USB.

Software serial and hardware serial - Most microcontrollers have hardware designed to communicate with other serial devices. Software serial ports use a pin-change interrupt system to communicate. There is a built-in library for Software Serial communication. Software serial is used by the processor to simulate extra serial ports. The only drawback with software serial is that it requires more processing and cannot support the same high speeds as hardware serial.

SPI - SS/SCK/MISO/MOSI pins are the dedicated pins for SPI communication. They can be found on digital pins 10-13 of the Arduino Uno and on the ICSP headers.

What is SPI?

Serial Peripheral Interface (SPI) is a serial data protocol used by microcontrollers to communicate with one or more external devices in a bus like connection. The SPI can also be used to connect 2 microcontrollers. On the SPI bus, there is always one device that is denoted as a Master device and all the rest as Slaves. In most cases, the microcontroller is the Master device. The SS (Slave Select) pin determines which device the Master is currently communicating with.

SPI enabled devices always have the following pins:

- MISO (Master In Slave Out) - A line for sending data to the Master device
- MOSI (Master Out Slave In) - The Master line for sending data to peripheral devices
- SCK (Serial Clock) - A clock signal generated by the Master device to synchronize data transmission.

I2C - SCL/SDA pins are the dedicated pins for I2C communication. On the Arduino Uno they are found on Analog pins A4 and A5.

What is I2C?

I2C is a communication protocol commonly referred to as the “I2C bus”. The I2C protocol was designed to enable communication between components on a single circuit board. With I2C there are 2 wires referred to as SCL and SDA.

- SCL is the clock line which is designed to synchronize data transfers.
- SDA is the line used to transmit data.

Each device on the I2C bus has a unique address, up to 255 devices can be connected on the same bus.

Aref - Reference voltage for the analog inputs.

Interrupt - INT0 and INT1. Arduino Uno has two external interrupt pins.

External Interrupt - An external interrupt is a system interrupt that occurs when outside interference is present. Interference can come from the user or other hardware devices in the network. Common uses for these interrupts in Arduino are reading the frequency a square wave generated by encoders or waking up the processor upon an external event.

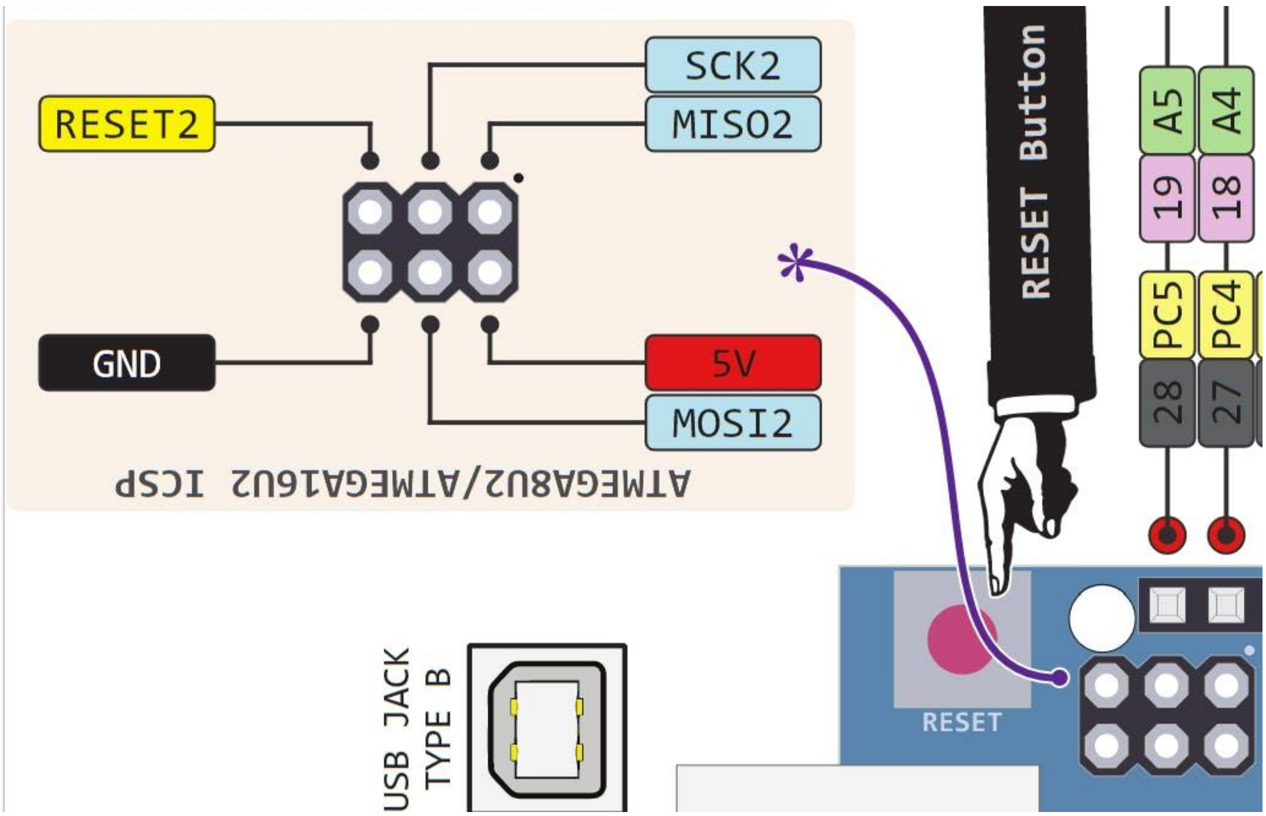
Arduino has two forms of interrupt:

- External
- Pin Change

There are two external interrupt pins on the ATmega168/328 called INT0 and INT1. both INT0 and INT1 are mapped to pins 2 and 3. In contrast, Pin Change interrupts can be activated on any of the pins.

Arduino Uno Pinout - ICSP Header

ICSP stands for In-Circuit Serial Programming. The name originated from In-System Programming headers (ISP). Manufacturers like Atmel who work with Arduino have developed their own in-circuit serial programming headers. These pins enable the user to program the Arduino boards' firmware. There are six ICSP pins available on the Arduino board that can be hooked to a programmer device via a programming cable.



Experiment No.: 1	Real-time monitoring and measurement of weather data	Date
Objectives of the experiment:	1. To read temperature and humidity data on the Arduino IDE. 2. To create loops for corresponding temperature and humidity values depending on problem statement	

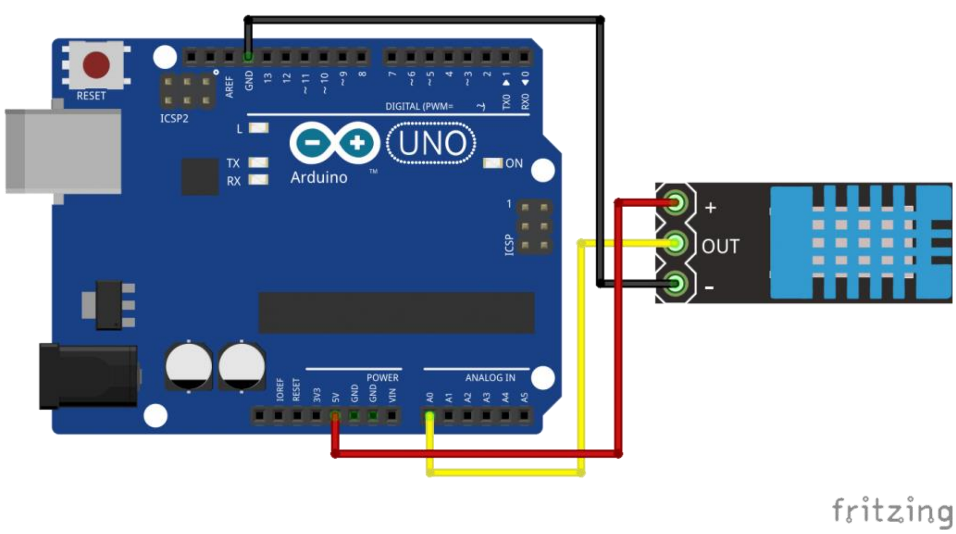
Apparatus / Software used:

S.No	Component Name	Quantity
1	DHT11 temperature sensor	**1**
2	Arduino Uno Board	**1**
3	Breadboard	**1**
4	Dupont wires	**10**
5	Arduino IDE (Software)	

Connection :

GND	GND
V_{cc}	5V
DHT11-data pin	A1

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
#include <dht.h>
dht DHT;
#define DHT11_PIN A1

float temperature,humidity;
float tem,hum;

unsigned int count = 1;        //For times count

void setup()
{
   Serial.begin(9600);
}

void loop()
{
      temperature = DHT.read11(DHT11_PIN);    //Temperature detection
      tem = DHT.temperature*1.0;
      humidity = DHT.read11(DHT11_PIN);
      hum = DHT.humidity*1.0;

      DHT.read11(DHT11_PIN);

      Serial.print(F("###########    "));
      Serial.print(F("NO."));
      Serial.print(count);
      Serial.println(F("    ###########"));
      Serial.println(F("The temperature and humidity :"));
```

```
	Serial.print(F("["));
	Serial.print(DHT.temperature);
	Serial.print(F("°C"));
	Serial.print(F(","));
	Serial.print(DHT.humidity);
	Serial.print(F("%"));
	Serial.print(F("]"));
	Serial.println("");
delay(10000);
	count++;

}
```

Results and Conclusion:

Applications of the experiment performed:

Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		

Marks

	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

Experiment No.: 2	Relay based real-time control of electrical equipment's.	Date
Objectives of the experiment:	1. To read relay status on the Arduino IDE. 2. To switch on and off the electrical equipment's using relay.	

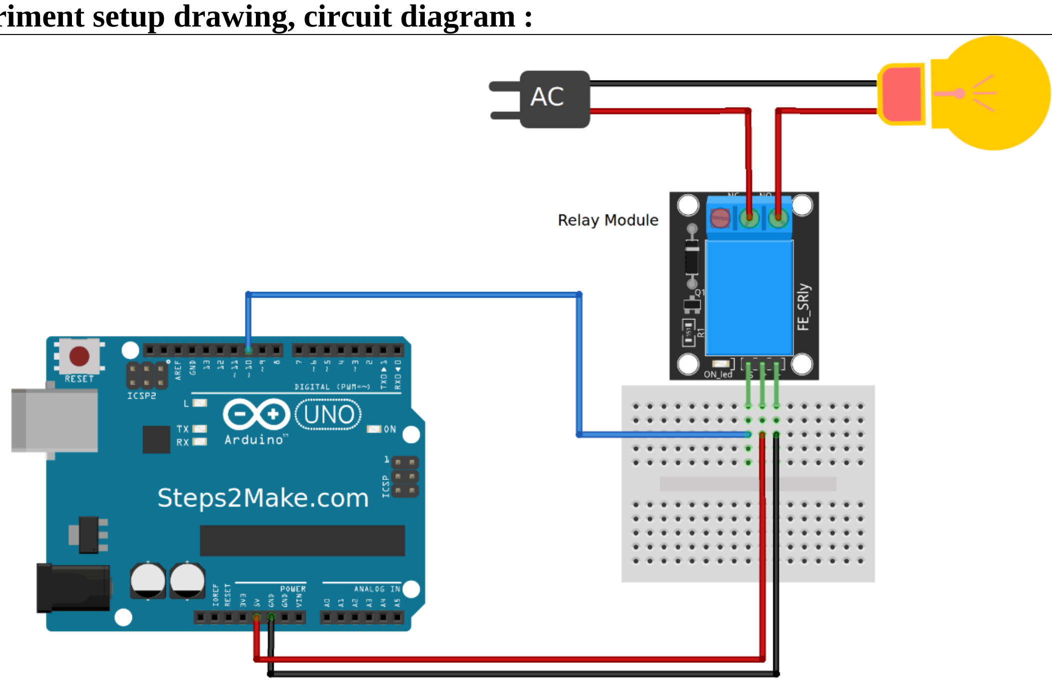
Apparatus / Software used:

S.No	Component Name	Quantity
1	Relay	**1**
2	Arduino Uno Board	**1**
3	Breadboard	**1**
4	Dupont wires	**10**
5	Arduino IDE (Software)	

Connection :

GND	GND
Vcc	5V
Relay	Digital pin 10

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
#define RELAY  10
void setup()
{
Serial.begin(9600);
   pinMode(RELAY, OUTPUT);
}
  void loop()
{
   digitalWrite(RELAY,0);           // Turns ON Relays 1
   Serial.println("Light ON");
   delay(2000);                                   // Wait 2 seconds

   digitalWrite(RELAY,1);          // Turns Relay Off
   Serial.println("Light OFF");
   delay(2000);
}
```

Results and Conclusion:

Applications of the experiment performed:

Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		

Marks		
	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

Experiment No.: 3	Water level monitoring with buzzer	Date
Objectives of the experiment:	1. To measure distance to monitor water level in a tank using Arduino IDE. 2. To create alarming system to ultrasonic sensor application.	

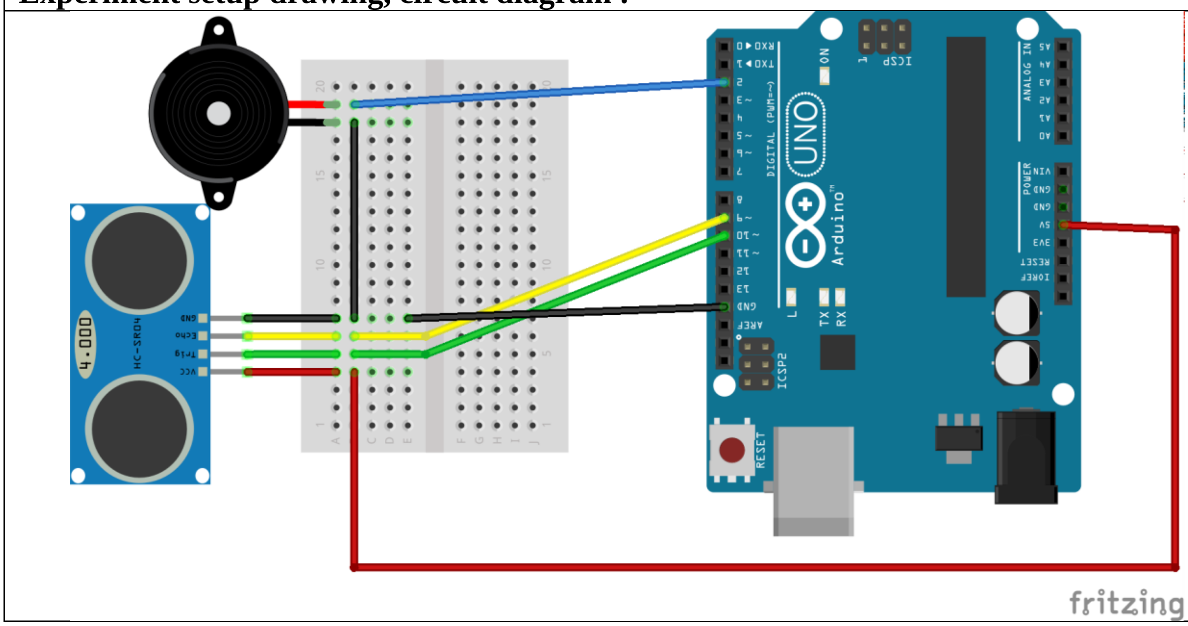
Apparatus / Software used:

S.No	Component Name	Quantity
1	Ultrasonic Sensor	1
2	Arduino Uno Board	1
3	Breadboard	1
4	Dupont wires	10
5	Arduino IDE (Software)	
6	Buzzer	1

Connection :

<table>
<tr><td>GND</td><td colspan="2">GND</td></tr>
<tr><td>Vcc</td><td colspan="2">5V</td></tr>
<tr><td>Buzzer</td><td colspan="2">Digital pin 2</td></tr>
<tr><td rowspan="2">Ultrasonic Sensor</td><td>Trig</td><td>10</td></tr>
<tr><td>Echo</td><td>9</td></tr>
</table>

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
int const trigPin = 10;
int const echoPin = 9;
int const buzzPin = 2;
void setup()
{
pinMode(trigPin, OUTPUT); // trig pin will have pulses output
pinMode(echoPin, INPUT); // echo pin should be input to get pulse width
pinMode(buzzPin, OUTPUT); // buzz pin is output to control buzzering
}
void loop()
{
// Duration will be the input pulse width and distance will be the distance to the obstacle in
centimeters
int duration, distance;
// Output pulse with 1ms width on trigPin
digitalWrite(trigPin, HIGH);
delay(1);
digitalWrite(trigPin, LOW);
// Measure the pulse input in echo pin
duration = pulseIn(echoPin, HIGH);
// Distance is half the duration devided by 29.1 (from datasheet)
distance = (duration/2) / 29.1;
// if distance less than 0.5 meter and more than 0 (0 or less means over range)
if (distance <= 50 && distance >= 0) {
// Buzz
digitalWrite(buzzPin, HIGH);
} else {
// Don't buzz
```

```
digitalWrite(buzzPin, LOW);
}
// Waiting 60 ms won't hurt any one
delay(60);
}
```

Results and Conclusion:

Applications of the experiment performed:

Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		

Marks		
	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

Experiment No.: 4	Automatic temperature controlling system	Date
Objectives of the experiment:	1. To read temperature and humidity data on the Arduino IDE. 2. To create loops for corresponding temperature and humidity values.	

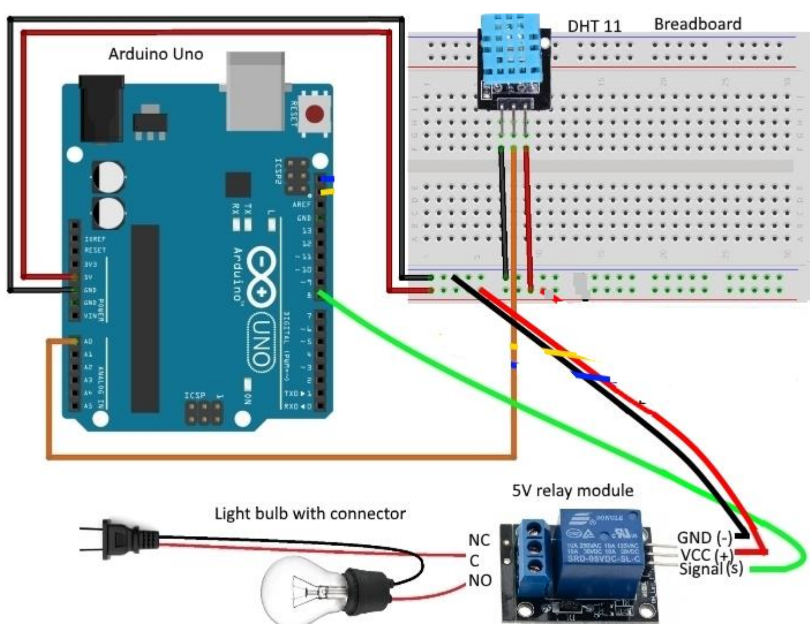
Apparatus / Software used:

S.No	Component Name	Quantity
1	DTH11 Sensor	**1**
2	Arduino Uno Board	**1**
3	Breadboard	**1**
4	Dupont wires	**10**
5	Arduino IDE (Software)	
6	Relay	**1**

Connection :

GND	GND
Vcc	5V
Relay	8
DHT11 Sensor	A0

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
#include <dht.h>
dht DHT;
#define DHT11_PIN A0
#define RELAY 8

float temperature,humidity;
float tem,hum;

unsigned int count = 1;       //For times count
void setup()
{
   Serial.begin(9600);
pinMode(RELAY, OUTPUT);
}

void loop()
{
      temperature = DHT.read11(DHT11_PIN);    //Temperature detection
      tem = DHT.temperature*1.0;
      humidity = DHT.read11(DHT11_PIN);
      hum = DHT.humidity*1.0;

      DHT.read11(DHT11_PIN);

      Serial.print(F("###########    "));
      Serial.print(F("NO."));
      Serial.print(count);
      Serial.println(F("    ###########"));
      Serial.println(F("The temperature and humidity :"));
      Serial.print(F("["));
      Serial.print(DHT.temperature);
```

```
      Serial.print(F("°C"));
      Serial.print(F(","));
      Serial.print(DHT.humidity);
      Serial.print(F("%"));
      Serial.print(F("]"));
      Serial.println("");
if (tem >= 23   )
 {
digitalWrite(RELAY,0);
}
else
{
digitalWrite(RELAY,1);
}
delay(10000);
      count++;

}
```

Results and Conclusion:

Applications of the experiment performed:

Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		
Marks		

	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

Experiment No.: 5	Flame detection and alerting system	Date
Objectives of the experiment:	1. To detect flame using Arduino IDE. 2. To create loops for corresponding IR value to trigger LED & Buzzer.	

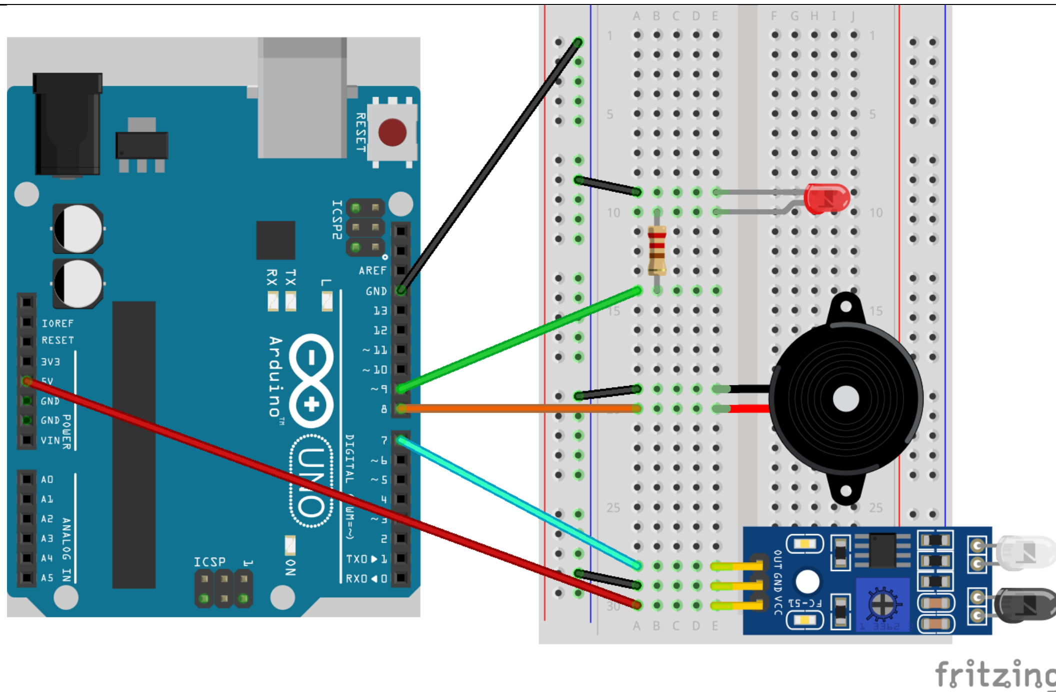
Apparatus / Software used:

S.No	Component Name	Quantity
1	IR Sensor	**1**
2	Arduino Uno Board	**1**
3	Breadboard	**1**
4	Dupont wires	**10**
5	Arduino IDE (Software)	
6	Buzzer	**1**
7	LED	**1**

Connection :

GND	GND
Vcc	5V
Buzzer	8
LED	9
IR DATA PIN	7 or A0

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
// IR Sensor with Arduino
//Digital code
int val = 0 ;
 void setup()
 {
 Serial.begin(9600); // sensor buart rate
 pinMode(7,INPUT);   // IR sensor output pin connected
 pinMode(9,OUTPUT);  // LED
 pinMode(8,OUTPUT);  // BUZZER
 }
 void loop()
```

```
 {
 val = digitalRead(2);  //  IR sensor output pin connected
 Serial.println(val);  // see the value in serial monitor in Arduino IDE
 delay(500);
 if(val == 1 )
 {
 digitalWrite(9,HIGH);  // LED ON
 digitalWrite(8,HIGH);  // BUZZER ON
 }
 else
 {
 digitalWrite(9,LOW);  // LED OFF
 digitalWrite(8,LOW);  // BUZZER OFF
 }
 }
```

Analog program

```
// IR Sensor with Arduino
void setup()
{
Serial.begin(9600); // sensor baud rate
pinMode(9,OUTPUT);  // LED
pinMode(8,OUTPUT);  // BUZZER
}
void loop()
{
int s1=analogRead(A0); //ANALOG PIN FOR SENSOR
Serial.println(s1);   // see the value in serial monitor in Arduino IDE
delay(100);
if(s1>500)
{
digitalWrite(9,HIGH);  // LED ON
digitalWrite(8,HIGH);  // BUZZER ON
}
else
{
digitalWrite(9,LOW);  // LED OFF
digitalWrite(8,LOW);  // BUZZER OFF
}
}
```

Results and Conclusion:

Applications of the experiment performed:

Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		

Marks		
	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

Experiment No.: 6	Automatic street lighting system	Date
Objectives of the experiment:	1. To measure illumination level using Arduino IDE. 2. To create loops for corresponding LDR value to trigger LED.	

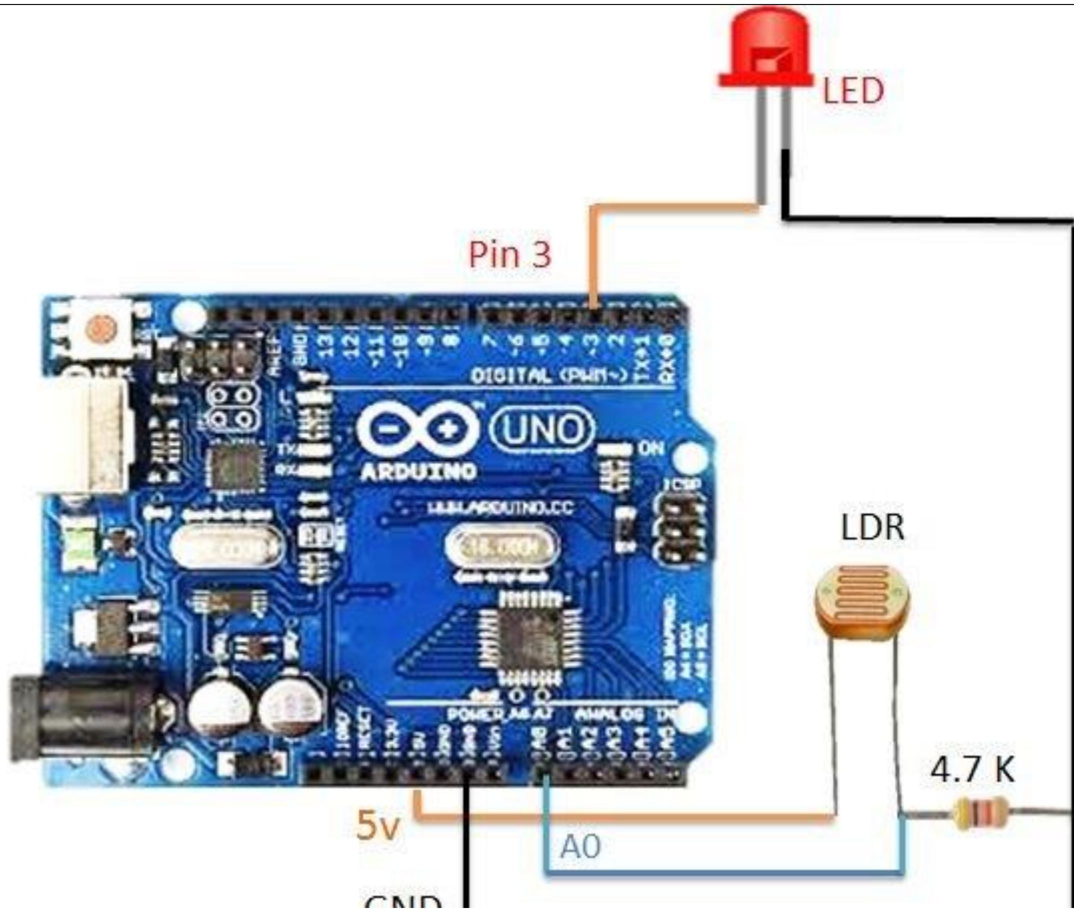
Apparatus / Software used:

S.No	Component Name	Quantity
1	LDR Sensor	**1**
2	Arduino Uno Board	**1**
3	Breadboard	**1**
4	Dupont wires	**10**
5	Arduino IDE (Software)	
6	Buzzer	**1**
7	LED	**1**

Connection :

GND	GND
Vcc	5V
LED	2
IR DATA PIN	7 or A0

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
// LDR Sensor with Arduino
//Digital code
int val = 0 ;
 void setup()
 {
```

```
 Serial.begin(9600); // sensor buart rate
 pinMode(7,INPUT);  // IR sensor output pin connected
 pinMode(3,OUTPUT);  // LED
 }
 void loop()
 {
 val = digitalRead(7);  //  IR sensor output pin connected
 Serial.println(val);  // see the value in serial monitor in Arduino IDE
 delay(500);
 if(val == 1 )
 {
 digitalWrite(3,HIGH);  // LED ON

 }
 else
 {
 digitalWrite(3,LOW);  // LED OFF
}
 }
```

Analog program

```
// IR Sensor with Arduino
void setup()
{
Serial.begin(9600); // sensor baud rate
pinMode(3,OUTPUT);  // LED

}
void loop()
{
int s1=analogRead(A0); //ANALOG PIN FOR SENSOR
Serial.println(s1);   // see the value in serial monitor in Arduino IDE
delay(100);
if(s1>200)
{
digitalWrite(3,HIGH);  // LED ON

}
else
{
digitalWrite(3,LOW);  // LED OFF

}
}
```

Results and Conclusion:

Applications of the experiment performed:		
Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		

Marks		
	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

Experiment No.: 7	**LED blinking operation using MATLAB**	Date
Objectives of the experiment:	1. To operate Arduino IDE using MATLAB. 2. To create loops for corresponding LED.	

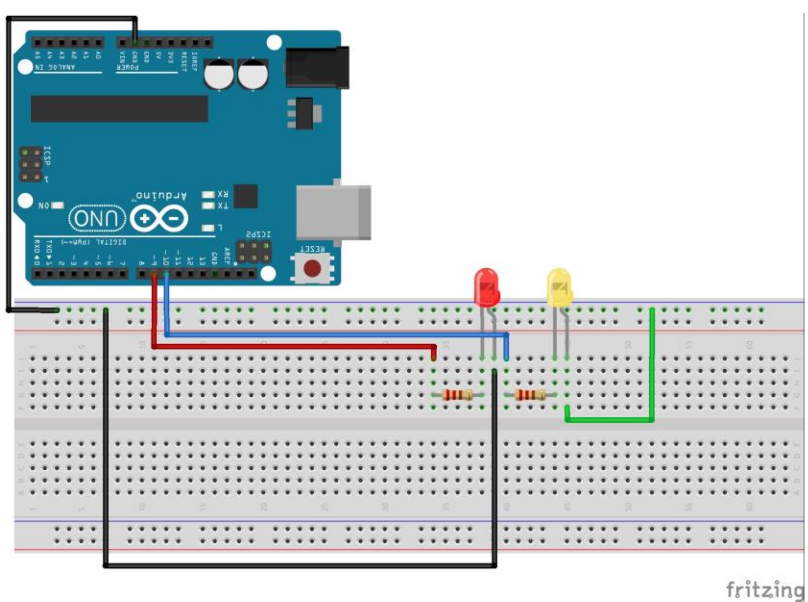
Apparatus / Software used:

S.No	Component Name	Quantity
1	LED	1
2	Arduino Uno Board	1
3	Breadboard	1
4	Dupont wires	10
5	Arduino IDE (Software)	
6	MATLAB 2024	

Connection :

GND	GND
Vcc	5V
LED	13

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
% create an arduino object

a = arduino('com19', 'uno');

% start the loop to blink led for 10 seconds

for i = 1:10

writeDigitalPin(a, 'D11', 1);
writeDigitalPin(a, 'D12', 0);


pause(0.5);

writeDigitalPin(a, 'D11', 0);
writeDigitalPin(a, 'D12', 1);


pause(0.5);

end

% end communication with arduino

clear a
```

Results and Conclusion:

Applications of the experiment performed:			
Observations :			
Calculations:			
Graph sheet (To be provided wherever required)			

Marks		
	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		

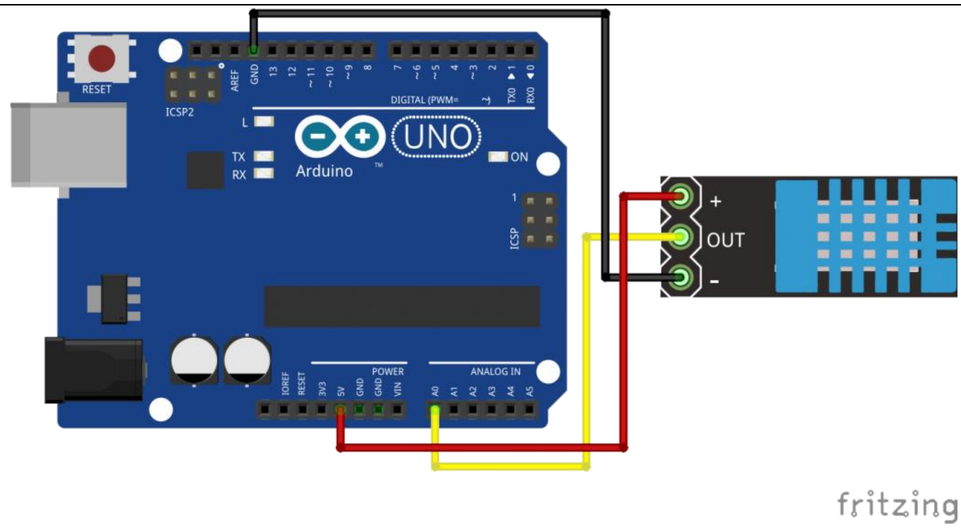
<table>
<tr><td>Experiment No.: 8</td><td colspan="2">MATLAB Data Logging, Analysis and Visualization: Plotting DHT11 Sensor readings on MATLAB</td><td>Date</td></tr>
<tr><td>Objectives of the experiment:</td><td colspan="3">1. To read temperature and humidity data on the MATLAB.
2. To achieve data Logging, analysis and visualization on MATLAB depending on problem statement</td></tr>
<tr><td colspan="4">Apparatus / Software used:</td></tr>
</table>

S.No	Component Name	Quantity
1	DHT11 temperature sensor	**1**
2	Arduino Uno Board	**1**
3	Breadboard	**1**
4	Dupont wires	**10**
5	Arduino IDE (Software)	
6	MATLAB 2024	

Connection :

GND	GND
V_{cc}	5V
DHT11-data pin	A1

Experiment setup drawing, circuit diagram :



Procedure / Algorithm:

Program:

```
#include <dht.h>
dht DHT;
#define DHT11_PIN A1

float temperature,humidity;
float tem,hum;

unsigned int count = 1;        //For times count

void setup()
{
   Serial.begin(9600);
```

```
}

void loop()
{
      temperature = DHT.read11(DHT11_PIN);    //Temperature detection
      tem = DHT.temperature*1.0;
      humidity = DHT.read11(DHT11_PIN);
      hum = DHT.humidity*1.0;

      DHT.read11(DHT11_PIN);

      Serial.print(F("###########    "));
      Serial.print(F("NO."));
      Serial.print(count);
      Serial.println(F("    ###########"));
      Serial.println(F("The temperature and humidity :"));
      Serial.print(F("["));
      Serial.print(DHT.temperature);
      Serial.print(F("℃"));
      Serial.print(F(","));
      Serial.print(DHT.humidity);
      Serial.print(F("%"));
      Serial.print(F("]"));
      Serial.println("");
delay(10000);
      count++;

}
```

MATLAB DATA ACQUISITION CODE

```
s = serial('COM18');
time=100;
i=1;
while(i<time)
fopen(s)
fprintf(s, 'Your serial data goes here')
out = fscanf(s)
Temp(i)=str2num(out(1:4));
 subplot(211);
 plot(Temp,'g');
 axis([0,time,20,50]);
title('Parameter: DHT11 Temperature');
xlabel('---> time in x*0.02 sec');
ylabel('---> Temperature');
grid
Humi(i)=str2num(out(5:9));
 subplot(212);
 plot(Humi,'m');
```

```
axis([0,time,25,100]);
title('Parameter: DHT11 Humidity');
xlabel('---> time in x*0.02 sec');
ylabel('---> % of Humidity ');
grid
fclose(s)
i=i+1;
drawnow;
end
delete(s)
clear s
```

Results and Conclusion:

Applications of the experiment performed:

Observations :		
Calculations:		
Graph sheet (To be provided wherever required)		

Marks		
	Maximum Marks	Marks Scored
Conducting the experiment	10	
Journal write-up	10	
Total	20	
Checked on		
Faculty Signature		